JAVA FILE HANDLING

File Class • File Permissions • FilePermission • FileDescriptor

---

Beginner-to-practical revision guide

Definitions, package details, constructors, method explanations, complete Java programs, output expectations, common mistakes and interview questions.

Learning objective: By the end of these notes, you should be able to explain what a pathname is, use the java.io.File API to manage file-system paths, understand Java's FilePermission representation, and obtain and use a FileDescriptor through an I/O stream.

Prepared as a structured study and practical-revision resource.

Contents

- 1. File-handling fundamentals
- 2. The java.io.File class
- 3. File constructors
- 4. File methods, grouped by purpose
- 5. Complete File class practical program
- 6. File permissions using java.io.File
- 7. The java.io.FilePermission class
- 8. The java.io.FileDescriptor class
- 9. Comparison of File, FilePermission and FileDescriptor
- 10. Common mistakes and practical checklist
- 11. Interview questions with short answers
- 12. Practice exercises

Version note: These notes describe the long-standing java.io APIs. Some permission behavior depends on the operating system and file system. The SecurityManager-based permission model is legacy: SecurityManager has been deprecated for removal since Java 17 and permanently disabled in Java 24.

1. File-handling fundamentals

1.1 What is a file?

A **file** is a named collection of data stored on a storage device, such as an SSD or hard disk. Examples include student.txt, photo.jpg, data.csv, Program.java and report.pdf.

1.2 What is a directory?

A **directory** (commonly called a folder) is a container used to organize files and other directories.

```
D:\
  └── Java
      └── FileHandling
          ├── student.txt
          ├── employee.txt
          └── images
              └── photo.jpg
```

1.3 What is a path?

A **path** identifies the location of a file or directory.

Path type	Meaning	Example
Absolute path	The complete location from a file-system root or drive.	D:\Java\FileHandling\student.txt
Relative path	A location interpreted from the current working directory.	src/student.txt

If the working directory is D:\Java\FileHandling, then src\student.txt refers to D:\Java\FileHandling\src\student.txt.

1.4 What does Java's File class do?

The **File** class represents an abstract pathname. It can inspect and manipulate file-system entries: create, delete, rename, test existence, inspect metadata, list directory entries and perform basic permission checks.

Critical distinction: new File("student.txt") creates a Java File object; it does not create a physical file. createNewFile() attempts to create the physical file.

1.5 What does File not do?

The File class does not read or write the contents of a file. For content I/O, use streams/readers/writers such as FileInputStream, FileOutputStream, FileReader, FileWriter, BufferedReader and BufferedWriter.

2. The java.io.File class

API definition: An abstract representation of file and directory pathnames.

Package: java.io
Import: import java.io.File;

2.1 Inheritance and interfaces

```
java.lang.Object
       |
   java.io.File
```

File directly extends Object and implements Serializable and Comparable<File>. Comparable allows File objects to be ordered by their pathnames.

```
File f1 = new File("apple.txt");
File f2 = new File("banana.txt");
System.out.println(f1.compareTo(f2));
```

compareTo() compares pathnames lexicographically. It does not compare file contents.

3. Constructors of the File class

The File class has four public constructors.

3.1 File(String pathname)

Creates a File instance by converting the supplied pathname string into an abstract pathname.

```
File f = new File("D:\\Java\\student.txt");
```

3.2 File(String parent, String child)

Creates a File instance from a parent pathname string and a child pathname string.

```
File f = new File("D:\\Java", "student.txt");
```

3.3 File(File parent, String child)

Creates a File instance from a parent File object and a child pathname string.

```
File parent = new File("D:\\Java");
File f = new File(parent, "student.txt");
```

3.4 File(URI uri)

Creates a File instance by converting a file URI into an abstract pathname. The URI must be an absolute, hierarchical file URI with a valid path.

```
import java.io.File;
import java.net.URI;

URI uri = URI.create("file:///D:/Java/student.txt");
File f = new File(uri);
System.out.println(f.getPath());
```

Constructor	Purpose
File(String pathname)	Uses an absolute or relative pathname.
File(String parent, String child)	Combines parent and child pathname strings.
File(File parent, String child)	Uses an existing parent File object.
File(URI uri)	Builds a File from a file URI.

4. File class methods

The methods are easier to learn when grouped by the job they perform.

4.1 Creation methods

Method	Purpose / behavior
createNewFile()	Creates a new empty file only if it does not already exist. Returns true if created; false if it already exists. Throws IOException.
mkdir()	Creates the directory named by this pathname. Does not create missing parent directories.
mkdirs()	Creates the directory and any missing parent directories.

```
File file = new File("student.txt");
boolean created = file.createNewFile();

File oneFolder = new File("JavaFiles");
boolean made = oneFolder.mkdir();

File nested = new File("JavaFiles/Programs/Advanced");
boolean allMade = nested.mkdirs();
```

4.2 Existence and type-checking methods

Method	Purpose
exists()	Tests whether the pathname exists.
isFile()	Tests whether it is an existing regular file.
isDirectory()	Tests whether it is an existing directory.
isHidden()	Tests whether the file system considers it hidden.
isAbsolute()	Tests whether the pathname is absolute.

```
File f = new File("student.txt");
System.out.println(f.exists());
System.out.println(f.isFile());
System.out.println(f.isDirectory());
System.out.println(f.isHidden());
System.out.println(f.isAbsolute());
```

4.3 Permission-checking methods

Method	Purpose
canRead()	Tests whether the application can read the file.
canWrite()	Tests whether the application can write to the file.
canExecute()	Tests whether the application can execute the file.

```
System.out.println(f.canRead());
System.out.println(f.canWrite());
System.out.println(f.canExecute());
```

These checks reflect the current process, operating-system rules and file-system behavior. They are not a guarantee that a later operation will succeed.

4.4 Path and file-information methods

Method	Purpose
getName()	Returns the final name component of the pathname.
getPath()	Returns the pathname string used to create the File object.
getAbsolutePath()	Returns the absolute pathname string.
getAbsoluteFile()	Returns a File object representing the absolute pathname.
getCanonicalPath()	Returns a canonical pathname, resolving . and .. and symbolic links where supported. Throws IOException.
getCanonicalFile()	Returns a File object representing the canonical pathname. Throws IOException.
getParent()	Returns the parent pathname string, or null if there is no parent.
getParentFile()	Returns the parent as a File object, or null if there is no parent.
length()	Returns file length in bytes. Directory length is not a reliable total of its contents.
lastModified()	Returns last-modified time in milliseconds since the Unix epoch; returns 0 if unavailable.
getFreeSpace()	Returns unallocated bytes on the partition containing the pathname.
getUsableSpace()	Returns bytes available to the Java application on that partition.
getTotalSpace()	Returns total size of the partition containing the pathname.

```
File f = new File("D:\\Java\\student.txt");
```

```
System.out.println(f.getName());
System.out.println(f.getPath());
System.out.println(f.getAbsolutePath());
System.out.println(f.getAbsoluteFile());
System.out.println(f.getCanonicalPath());
System.out.println(f.getCanonicalFile());
System.out.println(f.getParent());
System.out.println(f.getParentFile());
System.out.println(f.length());
System.out.println(new java.util.Date(f.lastModified()));
System.out.println(f.getFreeSpace());
System.out.println(f.getUsableSpace());
System.out.println(f.getTotalSpace());
```

getPath vs getAbsolutePath vs getCanonicalPath: getPath() preserves the pathname form supplied to the constructor; getAbsolutePath() makes it absolute; getCanonicalPath() returns a normalized canonical form and may resolve symbolic links.

4.5 Deletion and rename methods

Method	Purpose
delete()	Deletes the file or an empty directory. Returns true on success; false otherwise.
deleteOnExit()	Registers the file/directory for deletion when the JVM terminates normally.
renameTo(File dest)	Renames or moves the pathname. Behavior can be platform-dependent; prefer Files.move() for more controlled modern code.

```
File oldFile = new File("student.txt");
File newFile = new File("employee.txt");

boolean renamed = oldFile.renameTo(newFile);
System.out.println(renamed);

newFile.delete();
```

4.6 Metadata and permission-modification methods

Method	Purpose
setLastModified(long time)	Sets last-modified time in milliseconds since the epoch.
setReadOnly()	Marks the file read-only using the file system's supported mechanism.
setReadable(boolean readable)	Sets the owner's read permission.
setReadable(boolean readable, boolean ownerOnly)	Sets read permission; ownerOnly controls whether the request is limited to the owner.
setWritable(boolean writable)	Sets the owner's write permission.
setWritable(boolean writable, boolean ownerOnly)	Sets write permission with owner-only control.
setExecutable(boolean executable)	Sets the owner's execute permission.
setExecutable(boolean executable, boolean ownerOnly)	Sets execute permission with owner-only control.

```
f.setReadable(true);
f.setWritable(true);
f.setExecutable(true);

f.setWritable(false, true); // owner-only request
f.setReadOnly();
```

These methods return boolean values. Their exact effect depends on the operating system, file system, process privileges and security restrictions.

4.7 Directory-listing methods

Method	Purpose
list()	Returns names of entries in a directory as String[]. Returns null if the pathname is not a directory or an I/O error occurs.
list(FilenameFilter)	Returns entry names accepted by the supplied FilenameFilter.
listFiles()	Returns entries as File[].
listFiles(FilenameFilter)	Returns File objects accepted by a FilenameFilter.
listFiles(FileFilter)	Returns File objects accepted by a FileFilter.

```
File folder = new File("D:/Java");

String[] names = folder.list();
if (names != null) {
    for (String name : names) {
        System.out.println(name);
    }
}

File[] textFiles = folder.listFiles(
    (dir, name) -> name.endsWith(".txt")
);

File[] directories = folder.listFiles(
    file -> file.isDirectory()
);
```

A FilenameFilter receives the directory and entry name. A FileFilter receives a File object. Both are commonly used with lambda expressions.

4.8 Conversion and comparison methods

Method	Purpose
toURI()	Converts the pathname to a file URI.
toPath()	Converts the File object to a java.nio.file.Path.
compareTo(File)	Compares pathnames lexicographically.
equals(Object)	Checks whether two File objects represent the same abstract pathname.
hashCode()	Returns the hash code for the abstract pathname.
toString()	Returns the pathname string; also used implicitly when printing a File.

```
File a = new File("apple.txt");
File b = new File("banana.txt");

System.out.println(a.toURI());
System.out.println(a.toPath());
System.out.println(a.compareTo(b));
System.out.println(a.equals(b));
System.out.println(a.hashCode());
System.out.println(a.toString());
```

equals() does not determine whether two different pathnames refer to the same physical file through symbolic links.

4.9 Static methods and constants

Member	Purpose
createTempFile(String prefix, String suffix)	Creates an empty temporary file in the default temporary directory.
createTempFile(String prefix, String suffix, File directory)	Creates a temporary file in the specified directory.
listRoots()	Returns file-system roots, such as drive roots on Windows.
separator	Platform-specific directory separator string, such as backslash on Windows.
separatorChar	Directory separator as a char.
pathSeparator	Platform-specific separator between entries in a path list; semicolon on Windows.
pathSeparatorChar	Path-list separator as a char.

```
File temp = File.createTempFile("java", ".tmp");
File[] roots = File.listRoots();

for (File root : roots) {
    System.out.println(root);
}

System.out.println(File.separator);
System.out.println(File.pathSeparator);
```

The temporary-file prefix must contain at least three characters. Temporary files should be deleted when no longer needed, or registered with deleteOnExit() for simple short-lived programs.

Deprecated method: toURL() is deprecated. Prefer toURI().toURL() if a URL is specifically required.

5. Complete practical program – File class

This program creates a directory and file, inspects properties, lists directory entries, renames the file, converts it to Path and URI, and then deletes the file and directory.

```
import java.io.File;
import java.io.IOException;
import java.util.Date;

public class FileClassDemo {
    public static void main(String[] args) throws IOException {

        // 1. Create a directory
        File folder = new File("JavaFileDemo");

        if (folder.mkdir() || folder.exists()) {
            System.out.println("Directory is available");
        }

        // 2. Create a file inside the directory
        File file = new File(folder, "student.txt");

        if (file.createNewFile()) {
            System.out.println("File created successfully");
        } else {
            System.out.println("File already exists");
        }

        // 3. Check file properties
        System.out.println("Exists: " + file.exists());
        System.out.println("Is File: " + file.isFile());
        System.out.println("Is Directory: " + file.isDirectory());
        System.out.println("Is Hidden: " + file.isHidden());

        // 4. File information
        System.out.println("Name: " + file.getName());
        System.out.println("Path: " + file.getPath());
        System.out.println("Absolute Path: " + file.getAbsolutePath());
        System.out.println("Parent: " + file.getParent());
```

```
        System.out.println("Length: " + file.length());
        System.out.println("Last Modified: "
                + new Date(file.lastModified()));

        // 5. Permissions
        System.out.println("Readable: " + file.canRead());
        System.out.println("Writable: " + file.canWrite());
        System.out.println("Executable: " + file.canExecute());

        // 6. Directory listing
        System.out.println("Directory Contents:");
        File[] files = folder.listFiles();

        if (files != null) {
            for (File entry : files) {
                System.out.println(entry.getName());
            }
        }

        // 7. Rename the file
        File renamedFile = new File(folder, "employee.txt");

        if (file.renameTo(renamedFile)) {
            System.out.println("File renamed successfully");
        }

        // 8. Convert to Path and URI
        System.out.println("Path object: " + renamedFile.toPath());
        System.out.println("URI: " + renamedFile.toURI());

        // 9. Delete file and then empty directory
        if (renamedFile.delete()) {
            System.out.println("File deleted successfully");
        }

        if (folder.delete()) {
            System.out.println("Directory deleted successfully");
        }
    }
}
```

Expected output (typical first run)

```
Directory is available
File created successfully
Exists: true
Is File: true
Is Directory: false
Is Hidden: false
Name: student.txt
Path: JavaFileDemo\student.txt
Absolute Path: [your working directory]\JavaFileDemo\student.txt
Parent: JavaFileDemo
Length: 0
Last Modified: [date and time]
Readable: true
Writable: true
Executable: [platform-dependent]
Directory Contents:
student.txt
File renamed successfully
Path object: JavaFileDemo\employee.txt
URI: file:/...
File deleted successfully
Directory deleted successfully
```

The absolute path, URI, timestamp and permission results vary by machine. If the directory or file already exists, the creation messages will differ.

6. File permissions using java.io.File

6.1 What are file permissions?

File permissions determine which operations a user or process is allowed to perform on a file or directory.

Permission	For a regular file	For a directory
Read (r)	Read file contents.	List directory entries.
Write (w)	Modify file contents.	Create, delete or rename entries, subject to other permissions.
Execute (x)	Execute a program or script, if applicable.	Search or traverse the directory.

6.2 Check permissions

```
File file = new File("student.txt");

System.out.println("Read: " + file.canRead());
System.out.println("Write: " + file.canWrite());
System.out.println("Execute: " + file.canExecute());
```

6.3 Change permissions

Method	Example	Meaning
setReadable(boolean)	file.setReadable(true);	Requests read permission for the owner.
setWritable(boolean)	file.setWritable(false);	Requests removal of the owner's write permission.
setExecutable(boolean)	file.setExecutable(true);	Requests execute permission for the owner.
setReadOnly()	file.setReadOnly();	Marks the file read-only using supported file-system behavior.

The overloads that accept ownerOnly let the caller request that a change apply only to the owner. The methods return boolean to indicate whether the operation succeeded.

6.4 Complete permission demonstration

```
import java.io.File;
import java.io.IOException;

public class FilePermissionDemo {
    public static void main(String[] args) throws IOException {

        File file = new File("permission.txt");
        file.createNewFile();

        System.out.println("Initial Permissions");
        System.out.println("Read: " + file.canRead());
        System.out.println("Write: " + file.canWrite());
        System.out.println("Execute: " + file.canExecute());

        file.setReadable(true);
        file.setWritable(true);
        file.setExecutable(true);

        System.out.println("\nAfter Enabling Permissions");
        System.out.println("Read: " + file.canRead());
        System.out.println("Write: " + file.canWrite());
        System.out.println("Execute: " + file.canExecute());

        boolean changed = file.setWritable(false);
        System.out.println("\nWrite Permission Changed: " + changed);
        System.out.println("Write: " + file.canWrite());

        file.setWritable(true);
        file.setReadOnly();

        System.out.println("\nAfter setReadOnly()");
        System.out.println("Read: " + file.canRead());
        System.out.println("Write: " + file.canWrite());

        // Cleanup is best-effort and platform-dependent.
        file.setWritable(true);
        file.delete();
    }
}
```

Permission behavior is not identical on Windows, Linux and macOS. A process with elevated privileges may still be able to modify a file that appears non-writable to an ordinary process. Treat these methods as basic file-system permission operations, not a complete security boundary.

7. The java.io.FilePermission class

7.1 Definition and purpose

Definition: FilePermission represents access to a file or directory within Java's permission model.

Package: java.io
Import: import java.io.FilePermission;

Unlike File, FilePermission does not create, read, write or delete a file. It represents a permission target and a set of actions.

File	FilePermission
Represents a pathname.	Represents permission to perform specified actions on a pathname.
Provides file operations and metadata checks.	Provides permission comparison and action information.
Example: file.createNewFile().	Example: new FilePermission("student.txt", "read").

7.2 Constructors

Constructor 1: FilePermission(String path, String actions)

Creates a permission for the specified path and comma-separated actions.

```
FilePermission p =
    new FilePermission("student.txt", "read,write");
```

Constructor 2: FilePermission(String path, String actions, boolean directoryHint)

An additional constructor with a directory hint exists in the API for specifying directory interpretation. It is primarily intended for infrastructure use; ordinary application code should normally use the two-argument constructor.

```
FilePermission p =
    new FilePermission("D:/Java/*", "read", true);
```

7.3 Supported actions

Action	Meaning
read	Read file contents.
write	Write or modify file contents.
execute	Execute a file.
delete	Delete a file.
readlink	Read the target of a symbolic link.

Actions can be combined, for example read,write or read,write,delete.

7.4 FilePermission path patterns

Path pattern	Meaning
student.txt	A specific file path.
D:\Java*	Entries immediately inside the directory; not recursive.
D:\Java\-	Entries in the directory and its subdirectories recursively.
<>	All files and directories.

7.5 FilePermission methods

Method	Purpose
getActions()	Returns the canonical comma-separated action string.
implies(Permission p)	Returns true if this permission includes the requested permission.
equals(Object obj)	Tests whether two FilePermission objects are equal.
hashCode()	Returns the permission object's hash code.
newPermissionCollection()	Creates a suitable PermissionCollection for FilePermission objects.

getActions()

```
FilePermission p =
    new FilePermission("student.txt", "read,write");

System.out.println(p.getActions());
// read,write
```

implies(Permission p)

```
FilePermission broad =
    new FilePermission("student.txt", "read,write");

FilePermission narrow =
    new FilePermission("student.txt", "read");

System.out.println(broad.implies(narrow));
// true
```

equals() and hashCode()

```
FilePermission p1 =
    new FilePermission("student.txt", "read");
FilePermission p2 =
    new FilePermission("student.txt", "read");

System.out.println(p1.equals(p2)); // true
System.out.println(p1.hashCode());
```

newPermissionCollection()

```
import java.io.FilePermission;
import java.security.PermissionCollection;

FilePermission read =
    new FilePermission("student.txt", "read");
FilePermission write =
    new FilePermission("student.txt", "write");

PermissionCollection permissions =
    read.newPermissionCollection();

permissions.add(read);
permissions.add(write);

System.out.println(permissions.implies(
    new FilePermission("student.txt", "read")
));
// true
```

7.6 Important limitation

> Creating a FilePermission object does not change operating-system permissions. It is a permission representation used by Java's legacy security-policy architecture. The SecurityManager has been deprecated for removal since Java 17 and permanently disabled in Java 24. Do not build new application security around the old SecurityManager model.

For modern applications, use operating-system permissions and appropriate APIs. For POSIX permissions where supported, look at java.nio.file.Files and PosixFilePermissions.

8. The java.io.FileDescriptor class

8.1 Understand the descriptor first

When a program opens a file, the operating system establishes an open resource that the program can use for input or output. Unix-like systems commonly identify such resources with file descriptors; Windows uses handles. Java's FileDescriptor is an opaque representation of the underlying machine-specific resource.

8.2 API definition

Instances of FileDescriptor serve as an opaque handle to the underlying machine-specific structure representing an open file, an open socket, or another source or sink of bytes.

Package: java.io
Import: import java.io.FileDescriptor;

```
java.lang.Object
       |
java.io.FileDescriptor
```

8.3 Why is it useful?

An output stream such as FileOutputStream opens a file and internally maintains a FileDescriptor. You can obtain it with getFD(). One important operation on the descriptor is sync(), which requests that system buffers be synchronized with the underlying device.

```
FileOutputStream fos =
    new FileOutputStream("student.txt");

FileDescriptor fd = fos.getFD();
```

8.4 Constructor

FileDescriptor() creates a FileDescriptor object that initially represents an invalid descriptor.

```
FileDescriptor fd = new FileDescriptor();
System.out.println(fd.valid()); // false
```

Constructing a FileDescriptor does not open a file. In ordinary programs, obtain a valid descriptor from an I/O stream.

8.5 Static fields

Field	Meaning
FileDescriptor.in	Standard input descriptor, associated with System.in.
FileDescriptor.out	Standard output descriptor, associated with System.out.
FileDescriptor.err	Standard error descriptor, associated with System.err.

```
System.out.println(FileDescriptor.in.valid());
System.out.println(FileDescriptor.out.valid());
System.out.println(FileDescriptor.err.valid());
```

These are normally valid in a standard Java process, although environments that redirect or close standard streams can change the result.

8.6 Public methods

valid()

Tests whether the FileDescriptor represents a valid open resource.

```
FileDescriptor fd = new FileDescriptor();
System.out.println(fd.valid()); // false

FileOutputStream fos =
    new FileOutputStream("student.txt");

FileDescriptor opened = fos.getFD();
System.out.println(opened.valid()); // normally true

fos.close();
System.out.println(opened.valid()); // normally false
```

sync()

Forces the system buffers associated with the descriptor to synchronize with the underlying storage device. It throws SyncFailedException if synchronization fails.

```
import java.io.FileDescriptor;
import java.io.FileOutputStream;
import java.io.IOException;

public class SyncDemo {
    public static void main(String[] args) throws IOException {

        FileOutputStream fos =
            new FileOutputStream("student.txt");

        fos.write("Hello Java".getBytes());
```

```
        FileDescriptor fd = fos.getFD();
        fd.sync();

        System.out.println("Data synchronization requested");

        fos.close();
    }
}
```

8.7 flush() versus sync()

flush()	FileDescriptor.sync()
Flushes buffered output from a Java stream toward the underlying system.	Requests synchronization of system buffers with the underlying storage device.
Available on stream/writer APIs that support flushing.	Available on FileDescriptor.
Does not by itself guarantee physical persistence on storage.	Provides a stronger persistence request, but actual guarantees still depend on the OS, file system and hardware.

8.8 Complete FileDescriptor program

```
import java.io.FileDescriptor;
import java.io.FileOutputStream;
import java.io.IOException;

public class FileDescriptorDemo {
    public static void main(String[] args) throws IOException {

        FileDescriptor invalidFD = new FileDescriptor();
        System.out.println("New Descriptor Valid: "
                + invalidFD.valid());

        FileOutputStream fos =
                new FileOutputStream("descriptor.txt");

        FileDescriptor fd = fos.getFD();
        System.out.println("Opened Descriptor Valid: "
                + fd.valid());

        String data = "Learning Java FileDescriptor";
        fos.write(data.getBytes());

        fd.sync();
        System.out.println("Data synchronized successfully");

        fos.close();

        System.out.println("Descriptor Valid After Closing: "
                + fd.valid());

        System.out.println("Standard Input: "
                + FileDescriptor.in.valid());
        System.out.println("Standard Output: "
                + FileDescriptor.out.valid());
        System.out.println("Standard Error: "
                + FileDescriptor.err.valid());
    }
}
```

Typical output:

```
New Descriptor Valid: false
Opened Descriptor Valid: true
Data synchronized successfully
Descriptor Valid After Closing: false
Standard Input: true
Standard Output: true
Standard Error: true
```

The standard-stream results are typical, not universal. A closed or redirected standard stream may behave differently.

9. Comparison: File vs FilePermission vs FileDescriptor

Feature	File	FilePermission	FileDescriptor
Package	java.io	java.io	java.io
Represents	A pathname	Access rights for a pathname	Underlying open resource handle
Creates/deletes files	Yes, via methods	No	No
Checks basic access	canRead/canWrite/canExecute	Permission-model checks	valid() checks descriptor validity
Reads/writes contents	No	No	No direct content API
Key methods	createNewFile, delete, listFiles	implies, getActions	valid, sync

A simple mental model

File: Which pathname are we talking about?
FilePermission: Which actions are represented as permitted for that pathname in Java's permission model?
FileDescriptor: Which opened low-level resource does an I/O stream use?

These classes are separate APIs. They do not automatically form a mandatory sequence, and FilePermission does not automatically enforce operating-system access.

10. Common beginner mistakes

- Thinking new File("a.txt") creates a physical file. It only creates a Java object representing a pathname.
- Using mkdir() when parent directories do not exist. Use mkdirs() when missing parents must also be created.
- Confusing getPath(), getAbsolutePath() and getCanonicalPath().
- Thinking length() returns characters. It returns bytes for a regular file.
- Assuming delete() can delete a non-empty directory.
- Assuming setReadOnly() provides complete protection against every possible modification.
- Assuming a FilePermission object changes the file's operating-system permissions.
- Thinking FileDescriptor is a replacement for FileInputStream or FileOutputStream.
- Failing to check whether list() or listFiles() returned null.
- Assuming sync() guarantees persistence regardless of hardware or file-system behavior.

11. Interview questions with short answers

1. What is the File class?

It is an abstract representation of file and directory pathnames in java.io.

2. Does new File("a.txt") create a file?

No. It creates a Java object representing the pathname. createNewFile() attempts physical creation.

3. What is the difference between mkdir() and mkdirs()?

mkdir() creates one directory; mkdirs() also creates missing parent directories.

4. What is the difference between exists() and isFile()?

exists() checks whether the pathname exists; isFile() checks whether it is an existing regular file.

5. What is the difference between getPath() and getAbsolutePath()?

getPath() returns the pathname form used by the object; getAbsolutePath() returns an absolute pathname.

6. What does length() return?

The file length in bytes; directory length is not a reliable total of its contents.

7. What is the difference between list() and listFiles()?

list() returns entry names as String[]; listFiles() returns entries as File[].

8. What does deleteOnExit() do?

Registers the pathname for deletion when the JVM terminates normally.

9. What is FilePermission?

A Java permission object representing specified actions on a file or directory pathname.

10. What are FilePermission actions?

read, write, execute, delete and readlink.

*11. What is the difference between * and - in FilePermission paths?*

* matches entries immediately in a directory; - includes descendants recursively.

12. Does FilePermission change OS permissions?

No. It represents permission information; it does not chmod or otherwise change OS permissions.

13. What is FileDescriptor?

An opaque Java handle representing an underlying machine-specific open resource.

14. Why is a new FileDescriptor invalid?

Its constructor creates an unassociated descriptor; it has not been connected to an open resource.

15. What does FileDescriptor.sync() do?

Requests synchronization of system buffers with the underlying storage device.

16. What is the difference between flush() and sync()?

flush() pushes Java-level buffered output to the underlying system; sync() requests system-buffer synchronization with storage.

17. How do you obtain a FileDescriptor?

For example, call getFD() on FileInputStream or FileOutputStream.

18. What happens to a descriptor when its stream is closed?

The associated resource is closed and the descriptor is normally no longer valid.

12. Practice exercises

Complete these exercises in IntelliJ IDEA. Do not just copy the earlier programs; write them yourself and test the result.

Exercise 1: Create a directory named JavaPractice using mkdir().

Hint: Verify it using exists() and isDirectory().

Exercise 2: Create five text files inside that directory.

Hint: Use createNewFile() and handle the boolean result.

Exercise 3: Display all entries in the directory.

Hint: Use listFiles() and handle null.

Exercise 4: Display only .txt files.

Hint: Use listFiles(FilenameFilter) or a lambda.

Exercise 5: Display each file's name, absolute path and size.

Hint: Use getName(), getAbsolutePath() and length().

Exercise 6: Rename one file.

Hint: Use renameTo() and inspect its boolean result.

Exercise 7: Check read, write and execute access.

Hint: Use canRead(), canWrite() and canExecute().

Exercise 8: Create a temporary file.

Hint: Use createTempFile() and remove it when finished.

Exercise 9: Write data and synchronize it.

Hint: Use FileOutputStream, getFD() and sync().

Exercise 10: Delete all created files and directories.

Hint: Delete files first, then delete the now-empty directory.

Self-check before moving to the next topic

- [ ] I can explain why a File object is not the same as a physical file.
- [ ] I can use all four File constructors.
- [ ] I can distinguish file creation, directory creation and directory listing methods.
- [ ] I can explain the main File metadata and permission methods.
- [ ] I can explain what FilePermission represents and why it does not change OS permissions.
- [ ] I can obtain a FileDescriptor from a stream and explain valid() and sync().
- [ ] I can write the File class practical program without copying it.

Next topic in the Java File Handling sequence

After this chapter, continue with **Byte Streams**: InputStream and OutputStream, followed by FileInputStream and FileOutputStream. Then study Character Streams, buffered streams, bridge streams, PrintWriter and Java NIO.

Study advice: Do not move on just because you have read the notes. Reproduce the practical programs from memory, deliberately cause common errors, and explain why they happen. That is what turns API familiarity into usable Java skill.